

ТЕХНИЧЕСКИЙ ПАСПОРТ И СХЕМАТИЧЕСКИЙ РАЗБОР МОДУЛЯ АДМИНИСТРИРОВАНИЯ (panel_users.py)

Настоящий документ содержит детальное архитектурное описание, профили безопасности и пошаговый структурный разбор всех исполнительных эндпоинтов модуля администрирования аккаунтов самой веб-панели и управления лимитами операторов.

МОДУЛЬ АДМИНИСТРИРОВАНИЯ ПАНЕЛИ (panel_users.py)

Эндпоинт обеспечивает атомарный ACID-накат и первоначальное конфигурирование учетных записей менеджеров панели СУБД с одновременной привязкой пулов разрешенных серверов и ролей.

--- Схема ACID-транзакции конфигурирования лимитов ---

ВХОДНОЙ ПАКЕТ ПРИВИЛЕГИЙ (PanelUserCreateRequest)

↓ Запуск ACID-сессии (conn.autocommit = False)

ЭТАП А: ПРОВЕРКА СУЩЕСТВОВАНИЯ И ИНЖЕКТ ХЭША ПОЛЬЗОВАТЕЛЯ

SELECT id -> INSERT (Режим нового) ИЛИ UPDATE (Режим модификации)

↓ Атомарная зачистка старой ИБ-матрицы оператора

ЭТАП Б: УДАЛЕНИЕ УСТАРЕВШИХ ЛИМИТОВ СТРОГО ПО USER_ID

DELETE FROM panel_user_server_access & panel_user_role_limits

↓ Веерный циклический инжект новых допусков

ЭТАП В: НАКАТ ОБНОВЛЕННОЙ МАТРИЦЫ ПРАВ (Циклы FOR)

1. **СЕРВЕРНЫЙ ДОПУСК:** Вставка разрешенных хостов СУБД в panel_user_server_access

2. **РОЛЕВОЙ ЗАБОР:** Вставка лимитов pg_* префиксов в panel_user_role_limits

↓ Запечатывание изменений в СУБД панели

ФИНАЛЬНЫЙ КОММИТ (`conn.commit()`) | ОТКАТ ПРИ ОШИБКЕ (`conn.rollback()`)

🔵 Профиль ИБ-защиты подсистемы конфигурации лимитов:

- **ACID-транзакционность матрицы прав:** На операциях сохранения лимитов принудительно отключается автокоммит (`conn.autocommit = False`). Каскадное удаление старых правил и веерный инъект новых записей в циклах `for` выполняются атомарно. При любом сетевом сбое или ошибке СУБД срабатывает `conn.rollback()`, защищая базу данных панели от полунакатанных или деформированных лимитов операторов.
- **Предотвращение сохранения небезопасных секретов:** Входной инпут пароля прогоняется через криптостойкий регулярный валидатор `validate_strong_password` (проверка длины ≥ 64 символа, наличия цифр, спецсимволов и разного регистра букв). На этапе сохранения пароль хешируется по 12-раундному алгоритму Bcrypt (`hashpw`), исключая хранение открытого текста.
- **Ликвидация веерного дублирования записей (Фикс ошибки 500):** Перед тем как вставить новые связи допусков серверов и ролей, система принудительно выполняет превентивное удаление старых записей по ключу `user_id`. Это на 100% ликвидирует риск падения СУБД по ошибкам нарушения уникальности индексов при повторном перевыпуске прав оператора.

--- Подробное описание эндпоинтов управления пользователями ---

- Реестр администраторов и менеджеров безопасности (GET `/api/admin/panel-users-list`) Роут закрыт сквозной проверкой ролей `RoleChecker(["admin", "Security_Manager", "auditor"])`. Использует суверенный оптимизированный SQL-запрос со вложенными подзапросами `string_agg(DISTINCT...)`. Это полностью исключает возникновение ошибки Декартова умножения строк при одновременной выгрузке массивов серверов и ролей. Сырые ошибки PostgreSQL маскируются, отдавая клиенту безопасный плоский JSON.
- Инлайн-смена паролей из строки таблицы мониторинга (PUT `/api/panel-users/change-password`) Доступ разрешен исключительно роли "admin". Входной инпут прогоняется через криптостойкий регулярный валидатор `validate_strong_password`. Пароль хешируется по 12-раундному алгоритму Bcrypt и заменяется в СУБД. Вызов логгера приведен к жесткому стандарту из 8 параметров для исключения `TypeError`.

--- Схема каскадного удаления и затвора самоликвидации системы ---

ЗАПРОС УДАЛЕНИЯ ПОЛЬЗОВАТЕЛЯ (DELETE `/api/panel-users/{user_id}`)



Вычитка имени по `user_id` до стирания

ИБ-ПРОВЕРКА НА КОРНЕВОЙ АККАУНТ (`username == "admin"`)



Разветвление логики безопасности

Попытка удалить "admin"

Мгновенная блокировка: HTTP 400 Крическая
защита

Любой другой менеджер

Исполнение DELETE CASCADE (успешное
удаление)

`utils.log_operation()` —► Сквозная фиксация инцидента или успеха в аудит-логе панели